

Construire un mini ChatGPT

Journal d'apprentissage complet avec nanoGPT

Brice Sommers | Mai 2026

PLAN

Étape 1 — Configurer votre environnement

- Installer Python, Pytorch et NanoGPT

Étape 2 — Comment fonctionne GPT (en termes simples)

- Tokenization, transformers et prédiction.

Étape 3 — Entraîner votre premier modèle

- Lancer NanoGPT en local sur les œuvres de Shakespear.

Étape 4 — Comprendre le code

- Revu rapide du code.

Étape 5 — Construire le modèle instruct (chat)

- Fine-tune le modèle.

Étape 1 — Configurer votre environnement

Objectif

Installer tous les outils nécessaires pour entraîner un modèle GPT sur un Mac Apple Silicon. Cela inclut Python, PyTorch avec la prise en charge de MPS, ainsi que le dépôt nanoGPT.

1.1 Vérifier la version de Python

Commencez par vérifier que Python est installé et qu'il respecte la version minimale requise (3.10+).

Commande :

```
python3 --version
```

1.2 Créer un environnement Conda

Au lieu d'installer les paquets à l'échelle du système (ce que macOS bloque), nous créons un environnement Conda isolé appelé 'nanogpt'. Le préfixe (base) dans le terminal a confirmé que Conda était déjà installé.

Commande :

```
conda create -n nanogpt python=3.11 -y
```

Activer l'environnement :

```
conda activate nanogpt
```

Le préfixe (nanogpt) confirme que l'environnement est actif. Tous les paquets installés à partir de ce moment sont isolés dans cet environnement.

1.3 Installer les paquets nécessaires

Installez PyTorch et toutes les bibliothèques de support nécessaires à l'entraînement.

Commande :

```
pip install torch numpy transformers datasets tiktoken wandb tqdm
```

1.4 Cloner le dépôt nanoGPT

Téléchargez la base de code nanoGPT d'Andrej Karpathy depuis GitHub.

Commande :

```
git clone https://github.com/karpathy/nanoGPT.git
cd nanoGPT
```

1.5 Vérifier la prise en charge du GPU (MAC)

Vérifiez que PyTorch peut détecter et utiliser le GPU Apple Silicon via le backend MPS (Metal Performance Shaders).

Commande :

```
python -c "import torch; print(torch.__version__);
print(torch.backends.mps.is_available())"
```

Résumé de l'étape 1

- Python 3.13.7 confirmé
- Environnement Conda 'nanogpt' créé avec Python 3.11
- PyTorch 2.12.0 et toutes les dépendances installés
- Dépôt nanoGPT cloné
- GPU Apple M1 confirmé comme prêt (MPS : True)

Pour reprendre le travail lors de prochaines sessions, exécutez toujours :

```
conda activate nanogpt
cd nanoGPT
```

Étape 2 — Comment fonctionne GPT (en termes simples)

Objectif

Construire un modèle mental du fonctionnement de GPT avant de toucher au code. Aucun calcul mathématique requis.

2.1 Tokens

GPT ne lit pas des lettres ni des mots complets. Il lit des tokens, c'est-à-dire des morceaux de texte. Les mots courants correspondent souvent à un seul token ; les mots rares sont divisés en plusieurs tokens. GPT-2 utilise un vocabulaire d'environ 50 000 tokens.

Chaque token est converti en nombre afin que le modèle puisse le traiter mathématiquement. Par exemple :

```
Bonjour -> 82681
,        -> 11
I        -> 40
am       -> 309
```

2.2 Prédiction

La seule tâche d'un modèle GPT est de prédire le token suivant à partir de tous les tokens précédents. Rien de plus. Il répète cette opération encore et encore, chaque nouveau token devenant une partie de l'entrée pour la prédiction suivante.

Exemple : à partir de 'The cat sat on the', le modèle prédit 'mat' avec une probabilité de 72 %. C'est toute l'intelligence de GPT. Répétez cela des milliards de fois sur suffisamment de texte, et quelque chose de remarquable émerge.

2.3 Le Transformer

Les tokens passent dans des couches empilées appelées blocs transformer. Chaque couche affine la compréhension :

- Couche 1 — motifs de base (séquences de lettres fréquentes)
- Couche 4 — relations entre les mots (quels mots vont ensemble)
- Couche 8 — grammaire et structure
- Couche 12 — prédiction finale

Chaque couche utilise l'attention, un mécanisme qui permet au modèle de regarder tous les autres tokens afin de décider lesquels sont pertinents. nanoGPT utilise 6 couches. GPT-4 en utilise des centaines.

2.4 Entraînement

L'entraînement consiste à montrer au modèle des exemples de texte de manière répétée et à l'ajuster progressivement pour qu'il s'améliore :

- Fournir du texte en entrée (Shakespeare, pages web, livres)
- Faire une prédiction pour le token suivant
- Mesurer à quel point il s'est trompé (la perte)
- Ajuster légèrement les poids pour réduire l'erreur
- Répéter des milliards de fois

Les poids sont des millions de nombres à l'intérieur du modèle. Après suffisamment d'entraînement, ces nombres encodent une quantité incroyable de connaissances sur le langage.

Étape 3 — Entraîner votre premier modèle

Objectif

Entraîner un modèle GPT au niveau caractère sur l'ensemble des œuvres de Shakespeare et générer du texte dans le style de Shakespeare.

3.1 Préparer le jeu de données Shakespeare

Commande :

```
python data/shakespeare_char/prepare.py
```

Le script a téléchargé toutes les œuvres de Shakespeare.

3.2 Entraîner le modèle

Entraînez le modèle avec le GPU M1 en spécifiant `--device=mps`. Le fichier de configuration crée un modèle de 10,65 M de paramètres avec 6 couches.

Commande :

```
python train.py config/train_shakespeare_char.py --device=mps
```

`--device=mps` (pour mac série M)

`--device=cuda` (pour machine avec carte graphique NVIDIA)

Paramètres clés de configuration :

```
n_layer = 6           # 6 couches transformer
n_head = 6             # 6 têtes d'attention
n_embd = 384           # taille des embeddings
block_size = 256       # fenêtre de contexte
max_iters = 5000       # étapes d'entraînement
batch_size = 64        # exemples par étape
```

Progression de l'entraînement (perte au fil du temps) :

```
étape 0 :   perte train 4.2874, perte val 4.2823 (bruit aléatoire)
étape 250 :  perte train 1.9795, perte val 2.0808 (sauvegarde du checkpoint)
étape 500 :  perte train 1.5572, perte val 1.7585 (sauvegarde du checkpoint)
étape 1000 : perte train 1.2817, perte val 1.5318 (sauvegarde du checkpoint)
étape 1500 : perte train 1.1506, perte val 1.4785 (sauvegarde du checkpoint)
étape 2000 : perte train 1.0525, perte val 1.4676 (sauvegarde du checkpoint)
étape 2500 : perte train 0.9537, perte val 1.4949
étape 3000 : perte train 0.8600, perte val 1.5223
étape 4000 : perte train 0.7029, perte val 1.6314
étape 5000 : perte train 0.6158, perte val 1.7013 (final)
```

3.3 Générer du texte avec le modèle entraîné

Commande :

```
python sample.py --out_dir=out-shakespeare-char --device=mps
```

--device=mps (pour mac série M)

--device=cuda (pour machine avec carte graphique NVIDIA)

Sortie d'exemple (extraits sélectionnés) :

```
Toi, et plein de succession de ce roi,  
Avait un lien qu'aucun homme ne pouvait voir dans ce monde,  
Condamné le roi. Comment, avec l'espoir qu'il est  
  
FRÈRE LAURENCE :  
Et tu étais notre paix, tes ailes vers les cieux.  
  
ROMÉO :  
Quoi ? Quand n'est-ce que mon pouvoir ? Homme ? Reviens cette fois loin de mon  
visage,  
Et jeté pour l'épée, pour me faire devenir cette lamentation :  
  
CAPULET :  
Comment la reine !  
  
NOURRICE :  
Eh bien ! N'avez-vous pas mis dans une bassesse ailée ?
```

Le modèle a réussi à apprendre les motifs du langage élisabéthain, la mise en forme théâtrale (PERSONNAGE : dialogue), les thèmes dramatiques et le style d'écriture de Shakespeare — entièrement à partir de zéro.

Étape 4 — Comprendre le code

Objectif

Comprendre ce que font réellement les deux fichiers principaux de nanoGPT. L'ensemble de la base de code ne compte qu'environ 600 lignes.

4.1 model.py (~300 lignes) — Le cerveau

Ce fichier définit l'architecture du réseau neuronal. Il contient quatre classes qui s'assemblent comme des briques Lego :

CausalSelfAttention

Le mécanisme d'attention. Il permet au modèle de regarder tous les tokens précédents pour décider lesquels sont pertinents pour la prochaine prédiction. 'Causal' signifie qu'il ne peut regarder que vers l'arrière, pas vers l'avant (sinon ce serait tricher).

MLP (perceptron multicouche)

Un réseau simple à deux couches qui traite chaque token après l'attention. On peut le voir comme le modèle qui 'réfléchit' à ce qu'il vient de remarquer.

Block

Une couche transformer complète. C'est simplement : attention + MLP empilés ensemble. nanoGPT empile 6 de ces blocs.

GPT

Le modèle complet. Il combine :

- Une couche d'embedding (convertit les tokens en vecteurs de nombres)
- Les 6 blocs transformer
- Une couche linéaire finale qui produit les probabilités du token suivant

4.2 train.py (~300 lignes) — Le professeur

Ce fichier exécute la boucle d'entraînement. La logique centrale de l'entraînement tient en 4 lignes :

```
X, Y = get_batch('train')      # récupérer un lot de texte
logits, loss = model(X, Y)      # faire des prédictions, mesurer
l'erreur
loss.backward()                 # déterminer ce qu'il faut corriger
optimizer.step()                # ajuster légèrement les poids
```

Tout le reste dans train.py relève de la gestion : sauvegarde des checkpoints toutes les 250 étapes, journalisation de la perte, ajustement du taux d'apprentissage au fil du temps, et chargement/sauvegarde des poids du modèle.

Étape 5 — Construire le modèle instruct (chat)

Objectif

Affiner le modèle sur des données de conversation afin qu'il apprenne à répondre comme un chatbot. C'est la même technique que celle utilisée pour transformer GPT-3 en ChatGPT.

Concept clé : modèle de base vs modèle instruct

Un modèle instruct utilise la même architecture, mais il est affiné sur des données de conversation formatées en paires Humain/Assistant. Il apprend le schéma : l'humain pose une question, l'assistant répond.

Le vrai ChatGPT a été construit exactement de cette manière : GPT-3 (base) -> affiné sur des conversations -> ChatGPT (instruct).

Nous allons construire un nouveau chatbot LLM NanoGPT appelé NanoGPT2 à partir du modèle de base GPT-2.

5.1 Installation

Clone le repo.

Commande:

```
git clone https://github.com/bricesommers/nanoGPT2
cd nanoGPT2
pip install -r requirements.txt
```

Créer un environnement virtuel.

Commande:

```
python3 -m venv .venv
source .venv/bin/activate
```

Installer les dépendances.

Commande:

```
pip install -r requirements.txt
```

5.2 Conversation Dataset

Un jeu de données de 50 conversations a été créé, couvrant des sujets tels que la science, la philosophie, l'IA, la programmation et la culture générale. Chaque échange utilise un format cohérent :

```
Human: What is artificial intelligence?
Assistant: Artificial intelligence is the field of building computer
systems that can perform tasks that normally require human
intelligence,
like understanding language, recognizing images, or making decisions.

Human: What is machine learning?
Assistant: Machine learning is a branch of AI where computers learn
from data instead of being explicitly programmed...
```

Le jeu de données est dans data/input.txt

5.3 Fine-tune

Utiliser le fichier input.txt pour effectuer le fine-tuning de GPT-2 afin de pouvoir converser avec lui.

Commande:

```
python finetune.py --data data/input.txt --epochs 5 --output out/
```

Options

Commande:

```
--data          path to your input.txt (default: data/input.txt)
--output        where to save the model (default: out/)
--epochs        number of training epochs (default: 5)
--lr            learning rate (default: 5e-5)
--batch_size    batch size (default: 2)
```

5.4 Chat

Chat avec le modèle.

Commande:

```
python chat.py --model out/
```

Options

Commande:

```
--model          path to fine-tuned model (default: out/)
--temperature    randomness of responses, 0.1-1.0 (default: 0.7)
--max_tokens     max length of each response (default: 150)
```